Lecture: Anatomy of a Full-Stack Web Application

Target Audience: Computer Science Graduates
Date: May 4, 2025
Goals:

1. Understand the distinct components and layers of a full-stack application.
2. Learn how clients find servers using DNS.
3. Grasp the roles and technologies of the client, server, and database.
4. Learn the fundamentals of HTTP as the communication backbone.
5. Understand the role of Load Balancers in scalability and availability.
6. Explore different types of Caching and Content Delivery Networks (CDNs).
7. **Learn how to handle long-running tasks asynchronously using Message Queues and Workers.**
8. Explore common authentication (AuthN) and authorization (AuthZ) patterns.
9. See how these pieces fit together in a typical request lifecycle.
10. Briefly touch upon modern architectural patterns.

1. Introduction: What is Full-Stack Development?

Welcome! As computer science graduates, you have a strong foundation in algorithms, data structures, and systems. Today, we bridge that knowledge to the practical world of building the web applications that millions use daily.

- **Definition:** Full-stack development encompasses the entire software stack required to deliver a web application. It involves working on:
  - **Frontend (Client-Side):** What the user sees and interacts with in their browser.
  - **Backend (Server-Side):** The engine that powers the application, handling logic and data.
  - **Database:** Where the application's data is stored and managed.
  - **DevOps (Often):** Aspects of deployment, hosting, and infrastructure.
- **Why is Architecture Crucial?** A well-designed architecture impacts:
  - **Scalability:** Can the application handle growth (users, data, features)?
  - **Maintainability:** How easy is it to fix bugs, update, or refactor?
  - **Performance:** How fast and responsive is the application?
  - **Security:** How well are data and logic protected?
  - **Testability:** Can components be tested independently?
  - **Team Velocity:** Does the architecture facilitate efficient development?

2. Finding the Server: The Domain Name System (DNS)

Before the client (browser) can even send an HTTP request to a server, it needs to

know the server's **IP address**. Humans prefer using domain names (like www.google.com), but computers communicate using numerical IP addresses (like 142.250.190.78 or an IPv6 address). DNS is the hierarchical and distributed naming system that translates human-readable domain names into machine-readable IP addresses.

- **How it Works (Simplified):**
  1. **User Input:** You type www.example.com into your browser.
  2. **Local Check:** Your browser/OS checks its local cache first. If visited recently, the IP might be stored there.
  3. **Resolver Query:** If not cached locally, your computer asks its configured DNS resolver (usually provided by your ISP or a public service like Google's 8.8.8.8 or Cloudflare's 1.1.1.1).
  4. **Recursive/Iterative Queries:** The resolver then performs a series of queries:
     - It asks a **Root Server** (.) where to find the TLD (Top-Level Domain) server for .com.
     - It asks the **TLD Server** (.com) where to find the authoritative **Name Server** for example.com. This server is configured by the domain owner and holds the actual DNS records.
     - It asks the **Authoritative Name Server** for example.com for the IP address (specifically, the A record for IPv4 or AAAA record for IPv6) associated with www.example.com.
  5. **Response to Resolver:** The authoritative name server responds with the IP address.
  6. **Response to Client:** The resolver sends the IP address back to your computer/browser. The resolver also caches this result for a period (defined by the record's TTL - Time To Live) to speed up future requests for the same domain.
  7. **HTTP Request:** Now that the browser has the IP address, it can initiate the TCP connection and send the HTTP request to that address.
- **Key Record Types:**
  - A: Maps a hostname to an IPv4 address.
  - AAAA: Maps a hostname to an IPv6 address.
  - CNAME (Canonical Name): Creates an alias from one hostname to another. The lookup continues with the canonical name.
  - MX (Mail Exchanger): Specifies mail servers responsible for accepting email for the domain.
  - NS (Name Server): Delegates a subdomain to a set of authoritative name servers.

    - TXT: Holds arbitrary text data, often used for verification (e.g., domain ownership, SPF records for email).
- **Importance:** DNS is fundamental infrastructure. Without it, we wouldn't be able to use domain names. DNS issues can make websites appear completely offline even if the servers are running. DNS configuration is also crucial for directing traffic, including pointing domains to load balancers or CDNs.

3. Core Components of a Web Application

Think of a standard web application as a system with three primary layers interacting, now reachable thanks to DNS:

a) The Client (Frontend)

- **Role:** Responsible for the User Interface (UI) and User Experience (UX). It renders information and captures user input.
- **Location:** Runs within the user's web browser (e.g., Chrome, Firefox, Edge, Safari).
- **Core Technologies:**
    - **HTML (HyperText Markup Language):** Defines the *structure* and semantic meaning of content.
      ```
      <h1>Welcome, User!</h1>
      <p>This is your dashboard.</p>
      <button id="loadDataBtn">Load Data</button>
      ```
    - **CSS (Cascading Style Sheets):** Defines the *presentation* and visual styling.
      ```
      h1 {
        color: #333;
        font-family: 'Arial', sans-serif;
      }
      button {
        padding: 10px 15px;
        background-color: blue;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
      }
      ```
    - **JavaScript (JS):** Adds *interactivity*, dynamic behavior, and communication capabilities.

```
// Example: Handle button click
const btn = document.getElementById('loadDataBtn');
btn.addEventListener('click', () => {
  console.log('Button clicked! Fetching data...');
  // Code to fetch data from server would go here
});
```

- **Modern Frontend Frameworks/Libraries:** React, Angular, Vue.js, Svelte.
- **Responsibilities:** Rendering UI, capturing input, client-side validation, making asynchronous requests, managing client-side state, handling routing (SPAs).

b) The Server (Backend)

- **Role:** The application's central logic hub. Processes requests, enforces business rules, interacts with data sources, and orchestrates responses.
- **Location & Scaling with Load Balancers:**
  - Runs on web servers (VMs, containers in the cloud).
  - **Single Server:** Simple, but a single point of failure and limited capacity.
  - **Multiple Servers & Load Balancers:** For handling significant traffic and ensuring high availability, applications are typically deployed across multiple identical server instances. A **Load Balancer** sits *in front* of these servers.
    - **Function:** It receives incoming client requests (often the DNS A/AAAA record for your domain points to the Load Balancer's IP address) and distributes them across the available backend server instances according to a specific algorithm (e.g., Round Robin, Least Connections, IP Hash).
    - **Benefits:**
      - **Scalability:** Easily add/remove server instances behind the load balancer to handle varying traffic loads (horizontal scaling).
      - **Availability/Fault Tolerance:** If one server instance fails, the load balancer detects this (via health checks) and stops sending traffic to it, routing requests to healthy instances instead. This prevents downtime.
      - **Performance:** Distributes load, preventing any single server from becoming overwhelmed.
      - **SSL Termination (Optional):** Load balancers can handle the decryption of HTTPS traffic, freeing up backend servers from this computationally intensive task.
    - **Types:** Cloud providers offer managed load balancers (e.g., AWS ELB, Google Cloud Load Balancing, Azure Load Balancer). Software load balancers like Nginx or HAProxy can also be used.

- **Core Technologies:** Node.js, Python, Java, Ruby, Go, C#, PHP + Frameworks (Express, Django, Spring Boot, Rails, Gin).
- **Web Server Software:** Nginx, Apache (often used *with* frameworks or as load balancers/reverse proxies).
- **Responsibilities:** Receiving requests (often via Load Balancer), AuthN/AuthZ, Business Logic, Server-Side Validation, Database Interaction, communicating with other services, formatting responses, error handling. **(Also, potentially queueing background tasks - see Section 7)**.
- Code Example (Conceptual Node.js/Express route):
  (No changes needed here, the code runs on each server instance behind the load balancer)

```
// server.js (simplified)
const express = require('express');
const app = express();
app.use(express.json()); // Middleware to parse JSON request bodies

// In-memory "database" for simplicity
let users = { '1': { id: '1', name: 'Alice', email: 'alice@example.com' } };

// GET endpoint to retrieve a user by ID
app.get('/api/users/:userId', (req, res) => {
  const userId = req.params.userId;
  const user = users[userId];
  if (user) {
    res.status(200).json(user);
  } else {
    res.status(404).json({ error: 'User not found' });
  }
});

// POST endpoint to create a new user
app.post('/api/users', (req, res) => {
  const { name, email } = req.body;
  if (!name || !email || !email.includes('@')) {
    return res.status(400).json({ error: 'Invalid input: Name and valid email are
required.' });
  }
  const newId = String(Date.now());
  const newUser = { id: newId, name, email };
```

```
    users[newId] = newUser;
    console.log('Created user:', newUser);
    res.status(201).json(newUser);
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

c) The Database (Persistence Layer)

- **Role:** Provides reliable, long-term storage and retrieval for the application's data.
- **Location:** Dedicated database servers or managed cloud services. Separate from application servers for scalability and resilience.
- **Types & Models:**
  - **Relational (SQL):** PostgreSQL, MySQL, SQL Server, Oracle. Strengths: ACID compliance, structured data.
    ```
    CREATE TABLE Products ( /* ... */ );
    INSERT INTO Products ( /* ... */ );
    SELECT name, price FROM Products WHERE price < 500;
    ```
  - **NoSQL:**
    - *Document:* MongoDB, Couchbase. Strengths: Flexible schema, semi-structured data.
    - *Key-Value:* Redis, Memcached. Strengths: Fast lookups, caching, session stores.
    - *Column-Family:* Cassandra, HBase. Strengths: Big Data, write-heavy workloads.
    - *Graph:* Neo4j, Neptune. Strengths: Highly connected data, relationships.
- **Responsibilities:** Storing/retrieving data, ensuring integrity, handling concurrency, backup/recovery, executing queries.

4. Communication: The HTTP Protocol

HTTP (HyperText Transfer Protocol) is the foundation of data communication, defining how clients request resources and servers respond.

- **Request-Response Model.**
- **Stateless:** Each request is independent. State management requires cookies, tokens, etc.
- **Key Components:**
  - **URL/URI:** Resource address (scheme, host, path, query string, fragment). The

host part is resolved via DNS.
    - **Request Methods:** GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD.
    - **Headers:** Metadata (e.g., Host, Accept, Content-Type, Authorization, Cookie, Cache-Control).
    - **Body:** Data payload (optional).
    - **Status Codes:** Indicate outcome (1xx, 2xx, 3xx, 4xx, 5xx).
- **Example HTTP Exchange (Fetching User Data):** *(No changes needed)*

5. Authentication (AuthN) & Authorization (AuthZ)

Managing user identity and permissions over stateless HTTP.

- **Authentication (AuthN):** Verifying identity.
- **Authorization (AuthZ):** Checking permissions.

Common Authentication Strategies:

a) Session-Based Authentication

- **How it works:** Login -> Server creates session ID -> Stores session data server-side -> Sends session ID in HttpOnly, Secure, SameSite cookie -> Browser sends cookie automatically -> Server looks up session.
- **Pros:** Central state, simpler concept, HttpOnly security.
- **Cons:** Server storage overhead, scaling challenges (sticky sessions/shared store needed behind load balancers), less API/mobile friendly.

b) Token-Based Authentication (e.g., JWT)

- **How it works:** Login -> Server creates signed JWT (Header, Payload, Signature) with claims -> Sends token to client -> Client stores token -> Client sends token in Authorization: Bearer <token> header -> Server verifies signature & claims (stateless verification).
- **Pros:** Stateless (server scalability), good for APIs/SPAs/mobile, cross-domain.
- **Cons:** Stale data possible, revocation harder, client storage security (XSS risk if in localStorage), secret key management.

Security Fundamentals: HTTPS, Password Hashing (bcrypt/Argon2 + salts), Server-Side Input Validation, Rate Limiting, Standard Libraries, Least Privilege, Update Dependencies.

6. Performance & Scalability Boosters: Caching & CDNs

Fetching data from a database or re-computing results for every request can be slow and expensive. Caching stores the results of operations or copies of content closer to

the user to speed up subsequent requests and reduce load on backend systems.

a) Browser Caching

- **What:** The user's browser stores local copies of static assets (CSS, JS, images, fonts) and even API responses based on HTTP caching headers sent by the server.
- **How:** The server includes headers like Cache-Control (e.g., Cache-Control: public, max-age=3600) and ETag or Last-Modified in its responses.
  - Cache-Control: Tells the browser *if* and for *how long* it can cache the response.
  - ETag / Last-Modified: Allow the browser to make conditional requests. If the content hasn't changed (checked using If-None-Match or If-Modified-Since request headers), the server responds with 304 Not Modified, and the browser uses its cached version, saving bandwidth.
- **Benefit:** Reduces latency for repeat visits, saves bandwidth, decreases server load for static assets.

b) Server-Side Caching / Application Caching

- **What:** The backend application stores frequently accessed data or computation results in a faster data store (often in-memory) than the primary database.
- **How:**
  - **In-Memory Caches:** Using data structures within the application server's memory (simple, but not shared across instances and lost on restart).
  - **External Cache Stores:** Using dedicated caching systems like **Redis** or **Memcached**. These are key-value stores optimized for speed. The application logic checks the cache first before querying the database or performing expensive computations. If data is found (cache hit), it's returned directly. If not (cache miss), the data is fetched/computed, stored in the cache, and then returned.
- **Common Uses:** Caching database query results, expensive computation results, rendered HTML fragments, user session data (alternative to DB session storage).
- **Challenge:** Cache invalidation – ensuring the cache is updated or removed when the underlying data changes. Strategies include Time-To-Live (TTL), explicit deletion on write, or more complex event-based invalidation.
- **Benefit:** Dramatically reduces database load, speeds up API response times, lowers computational cost.

c) Content Delivery Networks (CDNs)

- **What:** A geographically distributed network of proxy servers and their data

centers. They cache static content (images, CSS, JS, videos) at "edge locations" physically closer to end-users.

- **How:**
  1. You configure your application to serve static assets via CDN URLs (e.g., cdn.example.com/image.jpg instead of www.example.com/image.jpg). Often, DNS CNAME records point your asset subdomain to the CDN provider.
  2. When a user requests an asset for the first time in their region, the CDN edge server closest to them fetches the asset from your **origin server** (your main backend/storage).
  3. The edge server caches this asset locally.
  4. Subsequent requests for the same asset from users in that region are served directly and quickly from the nearby edge server's cache, without needing to contact your origin server.
- **Benefits:**
  - **Reduced Latency:** Users download assets from a nearby server, significantly improving load times.
  - **Reduced Origin Server Load:** Offloads traffic for static assets from your main servers.
  - **Increased Availability:** Distributes assets globally; if one edge server is down, others can serve the content.
  - **Scalability:** Handles massive traffic spikes for static content.
  - **Potential Cost Savings:** Bandwidth from CDNs can sometimes be cheaper than from your origin.
- **Providers:** Cloudflare, Akamai, AWS CloudFront, Google Cloud CDN, Fastly.

7. Handling Long-Running Tasks: Queues & Workers

(New Section)

Not all tasks initiated by a user request can or should be completed within the short lifespan of a typical HTTP request-response cycle (which usually times out after 30-60 seconds). Tasks like video processing, sending bulk emails, generating complex reports, or calling slow third-party APIs can take minutes or even hours. Trying to perform these **synchronously** within the web request leads to:

- **Bad User Experience:** The user stares at a loading spinner, potentially for a very long time, or the request times out entirely.
- **Resource Hogging:** The web server process handling the request remains tied up, unable to serve other users, potentially leading to resource exhaustion.
- **Poor Reliability:** If the server crashes or restarts during the long task, the work is

lost.

The Solution: Asynchronous Processing with Message Queues

The standard pattern is to decouple the long-running task from the initial web request using a message queue.

- **Components:**
  - **Message Queue (Broker):** A dedicated service that acts as an intermediary buffer for messages (tasks). Examples: RabbitMQ, Apache Kafka, Redis (using Lists or Streams), AWS SQS, Google Cloud Pub/Sub. It reliably stores messages until they are processed.
  - **Producer:** The component that creates messages and adds them to the queue. In our context, this is typically the **Backend Web Server**. When it receives a request to start a long task, it packages the necessary information into a message and sends it to the queue.
  - **Consumer (Worker):** A separate process (or pool of processes) running independently from the web server(s). Workers connect to the message queue, pull messages off it one by one, and execute the actual long-running task described in the message. Workers can be scaled independently of the web servers.
- **Workflow Example (Generating a Sales Report):**
  1. **User Request (Client -> Server):** User clicks "Generate Monthly Sales Report" in the frontend UI. A `POST` request is sent to `/api/reports`.
  2. **Task Enqueueing (Server -> Queue):** The backend server receives the request. It performs quick validation (e.g., check permissions, date range). Instead of generating the report directly, it creates a message like `{"report_type": "sales", "month": "2025-04", "user_id": "user123"}` and publishes it to a specific queue (e.g., `report_generation_queue`).
  3. **Immediate Response (Server -> Client):** The server immediately sends a response back to the client, typically a `202 Accepted` status code, indicating the request was accepted for processing but is not yet complete. The response might include a task ID: `{"message": "Report generation started.", "taskId": "report-xyz789"}`.
  4. **Worker Processing (Queue -> Worker):** One or more Worker processes are constantly monitoring the `report_generation_queue`. One worker picks up the message `{"report_type": "sales", ...}`.
  5. **Task Execution (Worker):** The worker executes the potentially long-running logic: querying the database for sales data, aggregating results, formatting the report (e.g., PDF, CSV). This happens completely outside the web request

lifecycle.

6. **Status Update/Notification (Worker -> DB/Notification Service):** Once the report is generated, the worker might:
    - Update a status record in the **Database** associated with `taskId: "report-xyz789"` to "completed".
    - Store the generated report file (e.g., in cloud storage like S3).
    - Send a notification to the user (e.g., push notification, email) or trigger another event.

- **Benefits:**
    - **Improved Responsiveness:** The web server responds to the user almost instantly.
    - **Increased Reliability:** If a worker crashes, the message often remains in the queue (or can be re-queued) to be picked up by another worker. Message queues often offer persistence options.
    - **Enhanced Scalability:** You can scale the number of web servers and worker processes independently based on load. If report generation is slow, add more workers without impacting web request handling capacity.
    - **Decoupling:** Web servers don't need to know the details of how tasks are executed, and workers don't need to know about web requests.
- **Frontend Handling of Asynchronous Tasks:** Since the initial response (`202 Accepted`) doesn't contain the final result, how does the frontend know when the task is done?
    - **Polling:** The simplest method. The frontend periodically sends requests to the backend (e.g., `GET /api/reports/status/report-xyz789`) asking for the task status. This is easy to implement but can be inefficient (many requests, potential delay in seeing completion).
    - **WebSockets:** A persistent, bidirectional communication channel is established between the client and server. When the worker completes the task and updates the status, the backend can *push* a notification directly to the relevant client(s) over the WebSocket connection in real-time. More complex but efficient for real-time updates.
    - **Server-Sent Events (SSE):** A persistent connection where the server can push updates *to* the client (unidirectional). Simpler than WebSockets if only server-to-client communication is needed for updates.
    - **User Notifications:** For very long tasks, simply telling the user "We'll email you when your report is ready" might be sufficient.

8. Putting It All Together: Request Lifecycle Example

(Renumbered from 7)

Let's trace a user logging in and then requesting a report (Token-Based Auth, considering DNS, LB, Caching, Queues):

1. **User Action (Client):** User types www.myapp.com, hits Enter.
2. **DNS Lookup:** Browser -> DNS -> gets **Load Balancer** IP.
3. **TCP Handshake:** Browser <-> Load Balancer.
4. **TLS Handshake:** Browser <-> Load Balancer (secure connection).
5. **HTTP Request (Initial Page Load):** Browser GET / -> Load Balancer.
6. **Load Balancer:** Forwards to a **Server Instance**.
7. **Server:** Renders login page (maybe using **Server-Side Cache**).
8. **Server Response:** HTML -> Load Balancer -> Browser (with `Cache-Control` headers).
9. **Browser Rendering & Asset Fetching:** Parses HTML, fetches CSS/JS/images from **Browser Cache** or **CDN**.
10. **User Action (Client):** User fills login form, clicks "Login".
11. **Client JS:** POST `/api/auth/login` -> Load Balancer.
12. **Load Balancer:** Forwards to Server Instance.
13. **Server (Backend):** Validates credentials against **Database**, generates JWT. **AuthN** succeeds.
14. **Server Response:** 200 OK with JWT -> Load Balancer -> Browser.
15. **Client JS:** Stores token, updates UI (shows dashboard).
16. **User Action (Client):** User clicks "Generate Report".
17. **Client JS:** Makes `POST /api/reports` request (to Load Balancer) with report parameters. Adds `Authorization` header with JWT.
18. **Load Balancer:** Forwards request to a Server Instance.
19. **Server (Backend):**
    - Auth Middleware verifies JWT.
    - Validates report parameters.
    - Creates a task message (e.g., `{"type": "sales_report", "params": {...}, "userId": "..."}`).
    - **Enqueues the message** onto the `report_queue` via a **Message Queue** client library.
    - Generates a unique `taskId`.
    - (Optional) Stores initial task status ("pending") in **Database** linked to `taskId`.
20. **Server Response:** Sends `202 Accepted` with the `taskId` back to Load Balancer -> Browser.
    - Response: Status: `202 Accepted`, Body: `{"message": "Report generation queued.", "taskId": "report-xyz789"}`.
21. **Client JS:** Receives `202` response. Updates UI (e.g., "Report is generating..."). Starts **Polling** `/api/reports/status/report-xyz789` every few seconds OR listens on a **WebSocket/SSE** connection for status updates pushed by the server.

22. **Worker Process:**
    - Pulls the message from the `report_queue`.
    - Executes the long report generation logic (querying **Database**, etc.).
    - Upon completion, updates the task status in the **Database** to "completed" and stores the report URL.
    - (If using WebSockets/SSE) Notifies the backend server component responsible for pushing updates.
23. **Backend (Status Check/Push):**
    - **(Polling):** When the client polls `/api/reports/status/report-xyz789`, the server checks the **Database** for the status. Once "completed", it returns the status and the report URL.
    - **(WebSocket/SSE):** The backend component receives notification (e.g., via an internal event or another queue) that the task is done and pushes the "completed" status and report URL to the client over the persistent connection.
24. **Client JS:** Receives the "completed" status and report URL. Updates the UI, providing a download link for the report.

9. Modern Architectural Considerations

(Renumbered from 8)
The basic Client-Server-Database model, enhanced by DNS, Load Balancers, Caching, and asynchronous processing via Queues/Workers, is the foundation. Modern applications often employ more complex patterns:

- **Monolith vs. Microservices:** (Description unchanged)
- **API Design (REST vs. GraphQL):** (Description unchanged)
- **Serverless Computing (FaaS):** (Description unchanged) Functions are excellent candidates for being **Workers** that process queue messages.
- **Containerization (Docker) & Orchestration (Kubernetes):** (Description unchanged) Kubernetes makes it easier to manage and scale deployments of web servers, workers, databases, and queue brokers.

10. Conclusion

(Renumbered from 9)
Understanding full-stack architecture involves seeing the web application as a system. DNS finds the entry point, Load Balancers distribute traffic, the client handles presentation, the server manages logic (using HTTP and AuthN/AuthZ), the database ensures persistence, Caching optimizes performance, and Message Queues with Workers enable reliable asynchronous processing of long tasks. Choosing the right technologies and patterns depends on specific requirements. Mastering these layers and how they connect is key to building effective, robust, performant, and responsive web applications.

Key Takeaways:

- Full-stack involves distinct Client, Server, and Database layers.
- DNS translates domain names to IP addresses.
- HTTP is the stateless communication protocol; AuthN/AuthZ add state management.
- Load Balancers enhance scalability and availability.
- Caching (Browser, Server-Side, CDN) improves performance and reduces load.
- **Use Message Queues and Workers for long-running tasks to improve responsiveness and reliability.**
- Choose technologies (languages, frameworks, DBs) appropriate for the task.
- Security (HTTPS, hashing, validation, permissions) must be integral.
- Modern architectures (Microservices, Serverless, Containers) address scale/complexity.

Questions?